Université Cadi Ayyad

Faculté des Sciences et Techniques

Marrakech

Module : Interface Homme-Machine (IHM)

Rapport de Travaux Pratiques

LAB 4

Application Java Swing

Réalisé par :

Zakia Azizi

1. Introduction et Objectifs

Ce rapport présente le travail réalisé dans le cadre du **Lab4** du module **Interface Homme-Machine (IHM)**. L'objectif est de concevoir et développer une application graphique en **Java Swing** permettant d'interagir avec des formes géométriques (carré, cercle, triangle) en appliquant des couleurs et des messages dynamiques. Le lab est structuré en 5 questions progressives.

Question	Objectif
Q1 – Structure	Organisation des classes et packages du projet Java
Q2 – Application	Fenêtre Swing avec GridBagLayout, composants et listeners
Q3 – Affichage	Dessin 2D de formes géométriques avec paintComponent
Q4 – Lambda	Refactorisation avec expressions lambda (ApplicationLambda)
Q5 – Améliorée	Version enrichie : couleurs RGB, tooltips, Graphics2D, antialiasing

2. Structure du Projet (Q1)

Le projet **lab4** est organisé sous le package **projetTest** et contient toutes les classes Java nécessaires à l'application, visibles dans l'IDE IntelliJ IDEA.

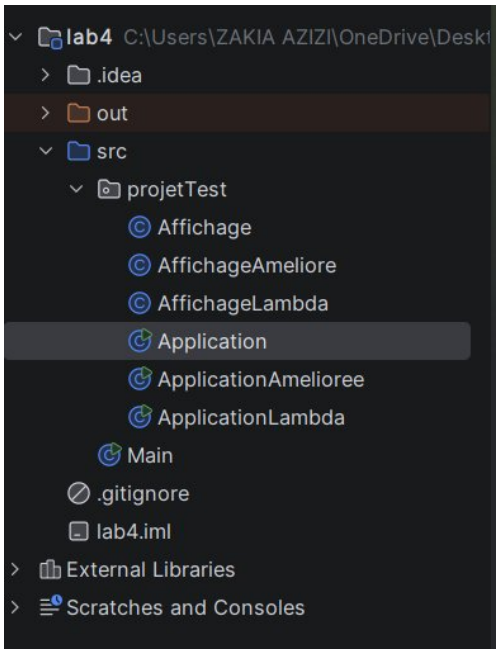

Figure 1 – Arborescence du projet lab4

Classe	Hérite de	Rôle
Affichage	JPanel	Panneau de dessin – version classique
AffichageAmeliore	JPanel	Dessin 2D amélioré (Graphics2D, antialiasing)
AffichageLambda	JPanel	Panneau de dessin – version lambda
Application	JFrame	Fenêtre principale – version classique
ApplicationAmelioree	JFrame	Fenêtre enrichie (tooltips, 5 couleurs, dialog)

ApplicationLambda	JFrame	Fenêtre refactorisée avec expressions lambda
Main	–	Point d'entrée du programme

3. Classe Application – Fenêtre Swing (Q2)

La classe **Application** étend **JFrame** et constitue la fenêtre principale. Elle utilise **GridBagLayout** pour positionner les composants : trois **JRadioButton** (Carré/Cercle/Triangle), un **JComboBox** de couleurs, une **JList** de messages, un **JTextField** et un bouton **Ajouter**. Les attributs m (forme) et message sont exposés via des getters utilisés par le panneau d'affichage.

```java
package projetTest;

import javax.swing.*;
import java.awt.*;

public class Application extends JFrame {
    private JRadioButton bouton1, bouton2, bouton3;  4 usages
    private JComboBox<String> selection;  4 usages
    private DefaultListModel<String> listModel;  7 usages
    private JList<String> textes;  4 usages
    private JTextField text_ajout;  4 usages
    private JButton ajouter;  3 usages
    private Affichage affiche;  8 usages
    private String m = "carre";  4 usages
    private String message = "Salam tout le monde";  2 usages

    public String getMessage() { return message; }  1 usage
    public String getForme() { return m; }  1 usage
    public String getCouleur() { return (String) selection.getSelectedItem(); }  1 usage

    public Application() {  1 usage
        setTitle("Atelier: Application en Java Swing");
        setSize( width: 600,  height: 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        JPanel contenu = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.BOTH;
        gbc.insets = new Insets( top: 5,  left: 5,  bottom: 5,  right: 5);
        affiche = new Affichage( a: this);
        affiche.setBackground(Color.WHITE);
```

Figure 2 – Attributs et getters de la classe Application

```java
public class Application extends JFrame {
    public Application() {  1 usage
        affiche = new Affichage( a: this);
        affiche.setBackground(Color.WHITE);
        gbc.gridx = 1;
        gbc.gridy = 0;
        gbc.gridwidth = 3;
        gbc.gridheight = 5;
        gbc.weightx = 0.7;
        gbc.weighty = 0.7;
        contenu.add(affiche, gbc);
        gbc.gridx = 0;
        gbc.gridy = 0;
        gbc.gridwidth = 1;
        gbc.gridheight = 1;
        gbc.weightx = 0.3;
        gbc.weighty = 0.1;
        contenu.add(new JLabel( text: "Forme"), gbc);
        bouton1 = new JRadioButton( text: "Carre",  selected: true);
        bouton2 = new JRadioButton( text: "Cercle");
        bouton3 = new JRadioButton( text: "Triangle");
        ButtonGroup group = new ButtonGroup();
        group.add(bouton1);
        group.add(bouton2);
        group.add(bouton3);
        gbc.gridy = 1;
        contenu.add(bouton1, gbc);
        gbc.gridy = 2;
        contenu.add(bouton2, gbc);
        gbc.gridy = 3;
        contenu.add(bouton3, gbc);
        gbc.gridy = 4;
```

Figure 3 – Construction GridBagLayout et RadioButtons

```java
public class Application extends JFrame {
    public Application() {  1 usage
        contenu.add(boutons, gbc);
        gbc.gridy = 4;
        contenu.add(new JLabel( text: "Couleur"), gbc);
        String[] couleurs = {"Bleu", "Vert", "Rouge"};
        selection = new JComboBox<>(couleurs);
        gbc.gridy = 5;
        contenu.add(selection, gbc);
        listModel = new DefaultListModel<>();
        listModel.addElement("Salam SIR FSTG 2020");
        listModel.addElement("Good new Year 2020");
        listModel.addElement("Good luck in Exams");
        listModel.addElement("Changez la couleur");
        textes = new JList<>(listModel);
        JScrollPane scroll = new JScrollPane(textes);
        gbc.gridx = 0;
        gbc.gridy = 6;
        gbc.gridwidth = 4;
        gbc.gridheight = 2;
        gbc.weighty = 0.4;
        contenu.add(scroll, gbc);
        JLabel labelAjout = new JLabel( text: "Ajouter un texte possible:");
        gbc.gridy = 8;
        gbc.gridheight = 1;
        gbc.weighty = 0.05;
        contenu.add(labelAjout, gbc);
        text_ajout = new JTextField();
        gbc.gridy = 9;
        contenu.add(text_ajout, gbc);
        ajouter = new JButton( text: "Ajouter");
        gbc.gridy = 10;
```

Figure 4 – JComboBox, JList, JTextField et JButton

```java
public class Application extends JFrame {
    public Application() {  1 usage
        ajouter = new JButton( text: "Ajouter");
        gbc.gridy = 10;
        contenu.add(ajouter, gbc);
        bouton1.addActionListener( ActionEvent e -> { m = "carre"; affiche.repaint(); });
        bouton2.addActionListener( ActionEvent e -> { m = "cercle"; affiche.repaint(); });
        bouton3.addActionListener( ActionEvent e -> { m = "triangle"; affiche.repaint(); });
        selection.addActionListener( ActionEvent e -> affiche.repaint());
        textes.addListSelectionListener( ListSelectionEvent e -> {
            if (!e.getValueIsAdjusting()) {
                String valeur = textes.getSelectedValue();
                if (valeur != null) {
                    message = valeur;
                    affiche.repaint();
                }
            }
        });
        ajouter.addActionListener( ActionEvent e -> {
            String texte = text_ajout.getText();
            if (!texte.isEmpty()) {
                listModel.addElement(texte);
                text_ajout.setText("");
            }
        });

        add(contenu);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Application().setVisible(true));
    }
```

Figure 5 – ActionListeners et ListSelectionListener

4. Classe Affichage – Dessin 2D (Q3)

La classe **Affichage** étend **JPanel** et redéfinit **paintComponent(Graphics g)** pour dessiner les formes. Elle reçoit une référence à **Application** pour lire la forme, la couleur et le message courants. Le dessin utilise :

- g.drawString() pour afficher le message en haut à gauche
- Un **switch** sur la couleur pour appliquer Color.BLUE/GREEN/RED
- Un **switch** sur la forme : fillRect, fillOval, fillPolygon

```java
public class Affichage extends JPanel {  2 usages
    public Affichage(Application a) {  // ← Constructeur avec Application  1 usage
        this.app = a;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawString(app.getMessage(), x: 20, y: 20);

        switch (app.getCouleur()) {
            case "Bleu": g.setColor(Color.BLUE); break;
            case "Vert": g.setColor(Color.GREEN); break;
            case "Rouge": g.setColor(Color.RED); break;
        }

        int centerX = getWidth() / 2;
        int centerY = getHeight() / 2 + 40;
        int size = 80;

        switch (app.getForme()) {
            case "carre":
                g.fillRect( x: centerX - size/2,  y: centerY - size/2, size, size);
                break;
            case "cercle":
                g.fillOval( x: centerX - size/2,  y: centerY - size/2, size, size);
                break;
            case "triangle":
                int[] x = {centerX, centerX - size/2, centerX + size/2};
                int[] y = {centerY - size/2, centerY + size/2, centerY + size/2};
                g.fillPolygon(x, y,  nPoints: 3);
                break;
```

Figure 6 – Méthode paintComponent de la classe Affichage

Résultats – Version classique (Q3) :

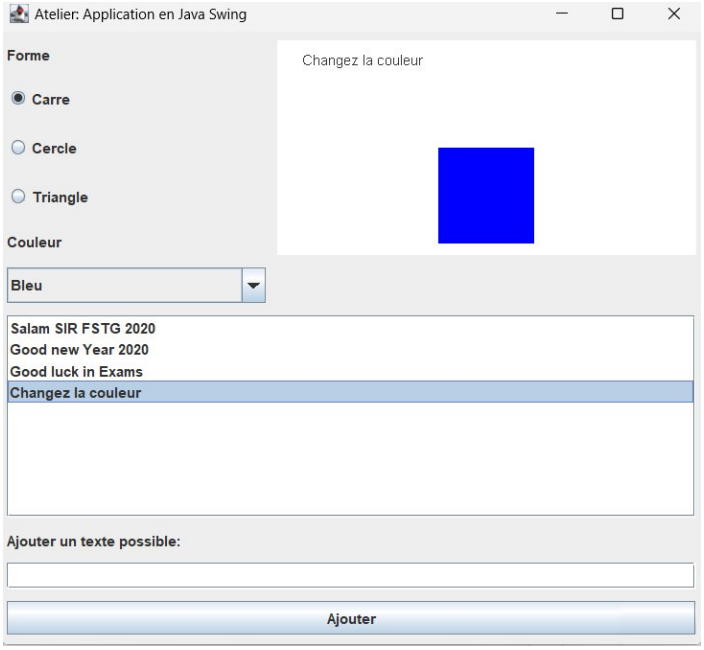

Figure 7 – Carré bleu, message « Changez la couleur »

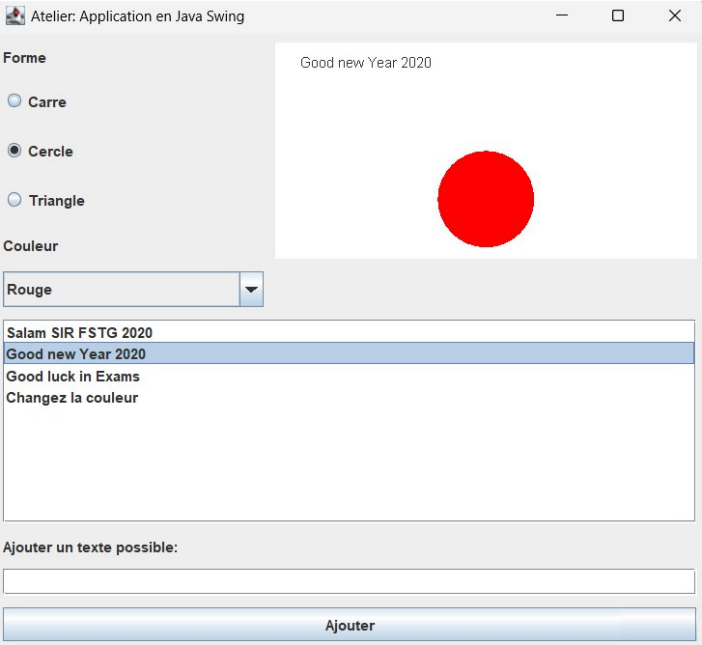

Figure 8 – Cercle rouge, message « Good new Year 2020 »

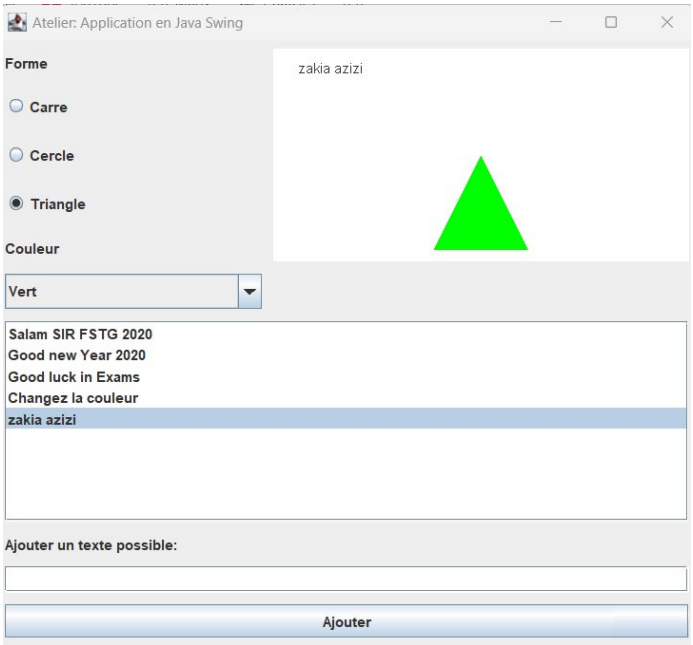

Figure 9 – Triangle vert, texte ajouté « zakia azizi »

5. Version Lambda – ApplicationLambda (Q4)

La question 4 consiste à refactoriser l'application en remplaçant toutes les classes anonymes par des **expressions lambda** Java 8. La structure reste identique à **Application**, seule la syntaxe des listeners change.

Version Classique	Version Lambda
`bouton1.addActionListener(new ActionListener(){ public void actionPerformed(...) { m = "carre"; affiche.repaint(); } });`	`bouton1.addActionListener( e -> { m = "carre"; affiche.repaint(); });`

```java
package projetTest;

import javax.swing.*;
import java.awt.*;

public class ApplicationLambda extends JFrame {
    private JRadioButton bouton1, bouton2, bouton3;  4 usages
    private JComboBox<String> selection;  4 usages
    private DefaultListModel<String> listModel;  7 usages
    private JList<String> textes;  4 usages
    private JTextField text_ajout;  4 usages
    private JButton ajouter;  3 usages
    private AffichageLambda affiche;  8 usages
    private String m = "carre";  4 usages
    private String message = "Salam tout le monde";  2 usages

    public String getMessage() { return message; }  1 usage
    public String getForme() { return m; }  1 usage
    public String getCouleur() { return (String) selection.getSelectedItem(); }  1 usage

    public ApplicationLambda() {  1 usage
        setTitle("Atelier: Application avec Lambda");
        setSize( width: 600,  height: 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        JPanel contenu = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.BOTH;
        gbc.insets = new Insets( top: 5,  left: 5,  bottom: 5,  right: 5);

        affiche = new AffichageLambda( a: this);
```

Figure 10 – Classe ApplicationLambda : attributs et constructeur

```java
public class ApplicationLambda extends JFrame {
    public ApplicationLambda() {  1 usage
        affiche = new AffichageLambda( a: this);
        affiche.setBackground(Color.WHITE);
        gbc.gridx = 1;
        gbc.gridy = 0;
        gbc.gridwidth = 3;
        gbc.gridheight = 5;
        gbc.weightx = 0.7;
        gbc.weighty = 0.7;
        contenu.add(affiche, gbc);

        gbc.gridx = 0;
        gbc.gridy = 0;
        gbc.gridwidth = 1;
        gbc.gridheight = 1;
        gbc.weightx = 0.3;
        gbc.weighty = 0.1;
        contenu.add(new JLabel( text: "Forme"), gbc);

        bouton1 = new JRadioButton( text: "Carre",  selected: true);
        bouton2 = new JRadioButton( text: "Cercle");
        bouton3 = new JRadioButton( text: "Triangle");

        ButtonGroup group = new ButtonGroup();
        group.add(bouton1);
        group.add(bouton2);
        group.add(bouton3);

        gbc.gridy = 1;
        contenu.add(bouton1, gbc);
```

Figure 11 – Panneau et RadioButtons dans ApplicationLambda

```java
public class ApplicationLambda extends JFrame {
    public ApplicationLambda() {  1 usage
        group.add(bouton3);

        gbc.gridy = 1;
        contenu.add(bouton1, gbc);
        gbc.gridy = 2;
        contenu.add(bouton2, gbc);
        gbc.gridy = 3;
        contenu.add(bouton3, gbc);

        bouton1.addActionListener( ActionEvent e -> { m = "carre"; affiche.repaint(); });
        bouton2.addActionListener( ActionEvent e -> { m = "cercle"; affiche.repaint(); });
        bouton3.addActionListener( ActionEvent e -> { m = "triangle"; affiche.repaint(); });

        gbc.gridy = 4;
        contenu.add(new JLabel( text: "Couleur"), gbc);
        String[] couleurs = {"Bleu", "Vert", "Rouge"};
        selection = new JComboBox<>(couleurs);
        gbc.gridy = 5;
        contenu.add(selection, gbc);

        selection.addActionListener( ActionEvent e -> affiche.repaint());

        listModel = new DefaultListModel<>();
        listModel.addElement("Salam SIR FSTG 2020");
        listModel.addElement("Good new Year 2020");
        listModel.addElement("Good luck in Exams");
        listModel.addElement("Changez la couleur");

        textes = new JList<>(listModel);
        JScrollPane scroll = new JScrollPane(textes);
```

Figure 12 – Listeners lambda pour boutons radio et couleur

```java
public class ApplicationLambda extends JFrame {
    public ApplicationLambda() {  1 usage

        textes = new JList<>(listModel);
        JScrollPane scroll = new JScrollPane(textes);
        gbc.gridx = 0;
        gbc.gridy = 6;
        gbc.gridwidth = 4;
        gbc.gridheight = 2;
        gbc.weighty = 0.4;
        contenu.add(scroll, gbc);

        // Lambda pour la sélection
        textes.addListSelectionListener( ListSelectionEvent e -> {
            if (!e.getValueIsAdjusting()) {
                String valeur = textes.getSelectedValue();
                if (valeur != null) {
                    message = valeur;
                    affiche.repaint();
                }
            }
        });

        JLabel labelAjout = new JLabel( text: "Ajouter un texte possible:");
        gbc.gridy = 8;
        gbc.gridheight = 1;
        gbc.weighty = 0.05;
        contenu.add(labelAjout, gbc);

        text_ajout = new JTextField();
        gbc.gridy = 9;
        contenu.add(text_ajout, gbc);
```

Figure 13 – Liste de messages et ListSelectionListener lambda

```java
public class ApplicationLambda extends JFrame {
    public ApplicationLambda() {  1 usage

        JLabel labelAjout = new JLabel( text: "Ajouter un texte possible:");
        gbc.gridy = 8;
        gbc.gridheight = 1;
        gbc.weighty = 0.05;
        contenu.add(labelAjout, gbc);

        text_ajout = new JTextField();
        gbc.gridy = 9;
        contenu.add(text_ajout, gbc);

        ajouter = new JButton( text: "Ajouter");
        gbc.gridy = 10;
        contenu.add(ajouter, gbc);

        // Lambda pour le bouton Ajouter
        ajouter.addActionListener( ActionEvent e -> {
            String texte = text_ajout.getText();
            if (!texte.isEmpty()) {
                listModel.addElement(texte);
                text_ajout.setText("");
            }
        });

        add(contenu);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ApplicationLambda().setVisible(true));
    }
```

Figure 14 – ActionListener lambda pour le bouton Ajouter

```java
package projetTest;

import java.awt.*;
import javax.swing.JPanel;

public class AffichageLambda extends JPanel {  2 usages
    private ApplicationLambda app;  // ← Type ApplicationLambda  4 usages

    public AffichageLambda(ApplicationLambda a) {  // ← Constructeur avec ApplicationLambda  1 usage
        this.app = a;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawString(app.getMessage(),  x: 20,  y: 20);

        switch (app.getCouleur()) {
            case "Bleu": g.setColor(Color.BLUE); break;
            case "Vert": g.setColor(Color.GREEN); break;
            case "Rouge": g.setColor(Color.RED); break;
        }

        int centerX = getWidth() / 2;
        int centerY = getHeight() / 2 + 40;
        int size = 80;

        switch (app.getForme()) {
            case "carre":
                g.fillRect( x: centerX - size/2,  y: centerY - size/2, size, size);
                break;
            case "cercle":
```

Figure 15 – Fin du constructeur et méthode main

```java
public class AffichageLambda extends JPanel {  2 usages
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawString(app.getMessage(), x: 20, y: 20);

        switch (app.getCouleur()) {
            case "Bleu": g.setColor(Color.BLUE); break;
            case "Vert": g.setColor(Color.GREEN); break;
            case "Rouge": g.setColor(Color.RED); break;
        }

        int centerX = getWidth() / 2;
        int centerY = getHeight() / 2 + 40;
        int size = 80;

        switch (app.getForme()) {
            case "carre":
                g.fillRect( x: centerX - size/2,  y: centerY - size/2, size, size);
                break;
            case "cercle":
                g.fillOval( x: centerX - size/2,  y: centerY - size/2, size, size);
                break;
            case "triangle":
                int[] x = {centerX, centerX - size/2, centerX + size/2};
                int[] y = {centerY - size/2, centerY + size/2, centerY + size/2};
                g.fillPolygon(x, y,  nPoints: 3);
                break;
        }
    }
}
```

Figure 16 – Classe AffichageLambda : paintComponent

Résultat – Version Lambda (Q4) :

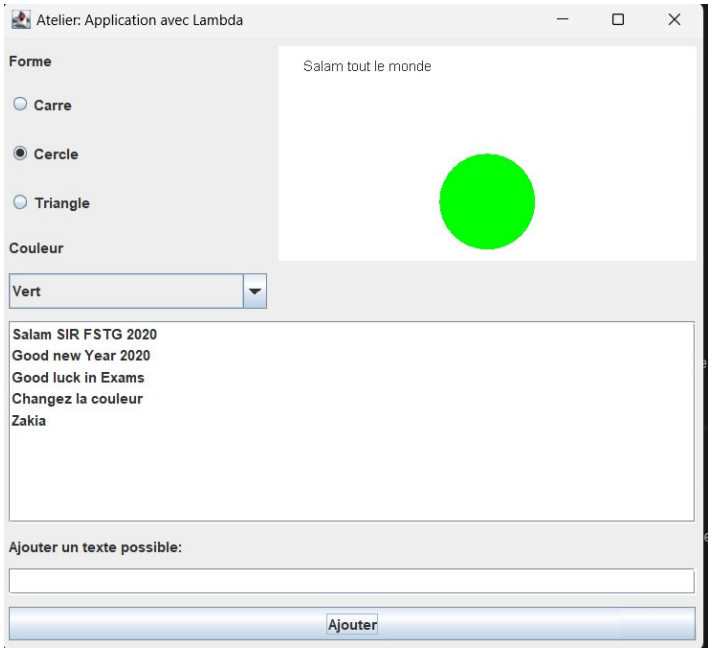

Figure 17 – ApplicationLambda : cercle vert, texte « Zakia »

6. Version Améliorée – ApplicationAmelioree (Q5)

La question 5 demande de créer une version enrichie de l'application avec plusieurs améliorations visuelles et fonctionnelles. La classe **ApplicationAmelioree** apporte les évolutions suivantes par rapport à la version de base :

Amélioration	Description
Fond personnalisé	Couleur de fond RGB(240,240,245) pour la fenêtre et le panneau
Bordure du panneau	LineBorder arrondie (épaisseur 2) autour du panneau Affichage

Icônes dans les labels	Émojis dans les labels (■ Forme, ● Cercle, ▲ Triangle, ■ Couleur)
ToolTips	Info-bulles sur tous les composants (RadioButtons, ComboBox, JList, JButton)
5 couleurs	Palette étendue : Bleu, Vert, Rouge, Orange, Magenta
Couleurs RGB précises	Couleurs définies via new Color(r,g,b) au lieu de Color.BLUE/GREEN/RED
Graphics2D + Antialiasing	Rendu lissé avec RenderingHints.VALUE_ANTIALIAS_ON
Carré arrondi	fillRoundRect (arcWidth=15, arcHeight=15) au lieu de fillRect
Texte ombré	Effet d'ombre sur le message (décalage de 2px)
Police personnalisée	Font "Segoe UI" Bold 14 pour les labels, Italic 16 pour le message
Bouton stylisé	Bouton Ajouter avec fond vert, texte blanc, police bold
Dialog validation	JOptionPane si le champ texte est vide lors du clic Ajouter
Centrage fenêtre	setLocationRelativeTo(null) pour centrer à l'écran

Code source – ApplicationAmelioree :

```java
package projetTest;

import javax.swing.*;
import javax.swing.border.LineBorder;
import java.awt.*;

public class ApplicationAmelioree extends JFrame {
    private JRadioButton bouton1, bouton2, bouton3;  5 usages
    private JComboBox<String> selection;  5 usages
    private DefaultListModel<String> listModel;  7 usages
    private JList<String> textes;  5 usages
    private JTextField text_ajout;  5 usages
    private JButton ajouter;  7 usages
    private AffichageAmeliore affiche;  9 usages
    private String m = "carre";  4 usages
    private String message = "Salam tout le monde";  2 usages

    public String getMessage() { return message; }  2 usages
    public String getForme() { return m; }  1 usage
    public String getCouleur() { return (String) selection.getSelectedItem(); }  1 usage

    public ApplicationAmelioree() {  1 usage
        setTitle("Atelier: Application Améliorée");
        setSize( width: 650,  height: 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);  // Centrer la fenêtre
        getContentPane().setBackground(new Color( r: 240,  g: 240,  b: 245));

        JPanel contenu = new JPanel(new GridBagLayout());
        contenu.setBackground(new Color( r: 240,  g: 240,  b: 245));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.BOTH;
```

Figure 18 – Attributs, imports et début du constructeur

```java
public class ApplicationAmelioree extends JFrame {
    public ApplicationAmelioree() {  1 usage
        JPanel contenu = new JPanel(new GridBagLayout());
        contenu.setBackground(new Color( r: 240,  g: 240,  b: 245));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.BOTH;
        gbc.insets = new Insets( top: 8,  left: 8,  bottom: 8,  right: 8);

        affiche = new AffichageAmeliore( a: this);
        affiche.setBackground(Color.WHITE);
        affiche.setBorder(new LineBorder(new Color( r: 100,  g: 100,  b: 150),  thickness: 2,  roundedCorners: true));

        gbc.gridx = 1;
        gbc.gridy = 0;
        gbc.gridwidth = 3;
        gbc.gridheight = 5;
        gbc.weightx = 0.7;
        gbc.weighty = 0.7;
        contenu.add(affiche, gbc);

        Font labelFont = new Font( name: "Segoe UI", Font.BOLD,  size: 14);

        gbc.gridx = 0;
        gbc.gridy = 0;
        gbc.gridwidth = 1;
        gbc.gridheight = 1;
        gbc.weightx = 0.3;
        gbc.weighty = 0.1;
        JLabel formeLabel = new JLabel( text: "📐 Forme");
        formeLabel.setFont(labelFont);
        contenu.add(formeLabel, gbc);
```

Figure 19 – Bordure, GridBagLayout et police des labels

```java
public class ApplicationAmelioree extends JFrame {
    public ApplicationAmelioree() {  1 usage
        bouton1 = new JRadioButton( text: "■ Carré",  selected: true);
        bouton2 = new JRadioButton( text: "● Cercle");
        bouton3 = new JRadioButton( text: "▲ Triangle");

        bouton1.setToolTipText("Dessine un carré");
        bouton2.setToolTipText("Dessine un cercle");
        bouton3.setToolTipText("Dessine un triangle");

        ButtonGroup group = new ButtonGroup();
        group.add(bouton1);
        group.add(bouton2);
        group.add(bouton3);

        gbc.gridy = 1;
        contenu.add(bouton1, gbc);
        gbc.gridy = 2;
        contenu.add(bouton2, gbc);
        gbc.gridy = 3;
        contenu.add(bouton3, gbc);

        bouton1.addActionListener( ActionEvent e -> { m = "carre"; affiche.repaint(); });
        bouton2.addActionListener( ActionEvent e -> { m = "cercle"; affiche.repaint(); });
        bouton3.addActionListener( ActionEvent e -> { m = "triangle"; affiche.repaint(); });

        gbc.gridy = 4;
        JLabel couleurLabel = new JLabel( text: "🎨 Couleur");
        couleurLabel.setFont(labelFont);
        contenu.add(couleurLabel, gbc);
```

Figure 20 – RadioButtons avec icônes et tooltips

```java
public class ApplicationAmelioree extends JFrame {
    public ApplicationAmelioree() {  1 usage

        String[] couleurs = {"Bleu", "Vert", "Rouge", "Orange", "Magenta"};
        selection = new JComboBox<>(couleurs);
        selection.setToolTipText("Choisissez la couleur de la forme");
        gbc.gridy = 5;
        contenu.add(selection, gbc);

        selection.addActionListener( ActionEvent e -> affiche.repaint());

        listModel = new DefaultListModel<>();
        listModel.addElement("💬 Salam SIR FSTG 2020");
        listModel.addElement("🎉 Good new Year 2020");
        listModel.addElement("🍀 Good luck in Exams");
        listModel.addElement("🌈 Changez la couleur");

        textes = new JList<>(listModel);
        textes.setToolTipText("Cliquez sur un message pour l'afficher");
        JScrollPane scroll = new JScrollPane(textes);
        scroll.setBorder(new LineBorder(new Color( r: 150,  g: 150,  b: 180),  thickness: 1,  roundedCorners: true));

        gbc.gridx = 0;
        gbc.gridy = 6;
        gbc.gridwidth = 4;
        gbc.gridheight = 2;
        gbc.weighty = 0.4;
        contenu.add(scroll, gbc);

        textes.addListSelectionListener( ListSelectionEvent e -> {
            if (!e.getValueIsAdjusting()) {
```

Figure 21 – JComboBox 5 couleurs, JList avec émojis

```java
public class ApplicationAmelioree extends JFrame {
    public ApplicationAmelioree() {  1 usage

        JLabel labelAjout = new JLabel( text: "✏️ Ajouter un texte :");
        labelAjout.setFont(labelFont);
        gbc.gridy = 8;
        gbc.gridheight = 1;
        gbc.weighty = 0.05;
        contenu.add(labelAjout, gbc);

        text_ajout = new JTextField();
        text_ajout.setToolTipText("Saisissez un nouveau message");
        gbc.gridy = 9;
        contenu.add(text_ajout, gbc);

        ajouter = new JButton( text: "✅ Ajouter");
        ajouter.setToolTipText("Ajoute le texte à la liste");
        ajouter.setBackground(new Color( r: 100,  g: 180,  b: 100));
        ajouter.setForeground(Color.WHITE);
        ajouter.setFont(new Font( name: "Segoe UI", Font.BOLD,  size: 12));
        gbc.gridy = 10;
        contenu.add(ajouter, gbc);

        ajouter.addActionListener( ActionEvent e -> {
            String texte = text_ajout.getText().trim();
            if (!texte.isEmpty()) {
                listModel.addElement("📝 " + texte);
                text_ajout.setText("");
            } else {
                JOptionPane.showMessageDialog( parentComponent: this,  message: "Veuillez saisir un texte !",  title: "Info", JOptionPane.I
            }
```

Figure 22 – Bouton Ajouter stylisé et dialog de validation

Code source – AffichageAmeliore :

```java
package projetTest;

import java.awt.*;
import javax.swing.JPanel;

public class AffichageAmeliore extends JPanel {  2 usages
    private ApplicationAmelioree app;  5 usages

    public AffichageAmeliore(ApplicationAmelioree a) {  1 usage
        this.app = a;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;

        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

        Font messageFont = new Font( name: "Segoe UI", Font.ITALIC,  size: 16);
        g2d.setFont(messageFont);
        g2d.setColor(new Color( r: 50,  g: 50,  b: 80));


        g2d.drawString(app.getMessage(),  x: 22,  y: 22);
        g2d.setColor(Color.BLACK);
        g2d.drawString(app.getMessage(),  x: 20,  y: 20);

        String couleur = app.getCouleur();
        switch (couleur) {
            case "Bleu": g2d.setColor(new Color( r: 66,  g: 133,  b: 244)); break;
            case "Vert": g2d.setColor(new Color( r: 52,  g: 168,  b: 83)); break;
```

Figure 23 – Graphics2D, antialiasing et effet ombré du texte

```java
public class AffichageAmeliore extends JPanel {  2 usages
    protected void paintComponent(Graphics g) {

        String couleur = app.getCouleur();
        switch (couleur) {
            case "Bleu": g2d.setColor(new Color( r: 66,  g: 133,  b: 244)); break;
            case "Vert": g2d.setColor(new Color( r: 52,  g: 168,  b: 83)); break;
            case "Rouge": g2d.setColor(new Color( r: 234,  g: 67,  b: 53)); break;
            case "Orange": g2d.setColor(new Color( r: 255,  g: 159,  b: 28)); break;
            case "Magenta": g2d.setColor(new Color( r: 200,  g: 70,  b: 200)); break;
            default: g2d.setColor(Color.BLUE);
        }

        int centerX = getWidth() / 2;
        int centerY = getHeight() / 2 + 30;
        int size = 90;

        switch (app.getForme()) {
            case "carre":
                g2d.fillRoundRect( x: centerX - size/2,  y: centerY - size/2, size, size,  arcWidth: 15,  arcHeight: 15);
                break;
            case "cercle":
                g2d.fillOval( x: centerX - size/2,  y: centerY - size/2, size, size);
                break;
            case "triangle":
                int[] x = {centerX, centerX - size/2, centerX + size/2};
                int[] y = {centerY - size/2, centerY + size/2, centerY + size/2};
                g2d.fillPolygon(x, y,  nPoints: 3);
                break;
        }
    }
}
```

Figure 24 – Switch couleurs RGB précises et dessin des formes

Résultats – Version Améliorée (Q5) :

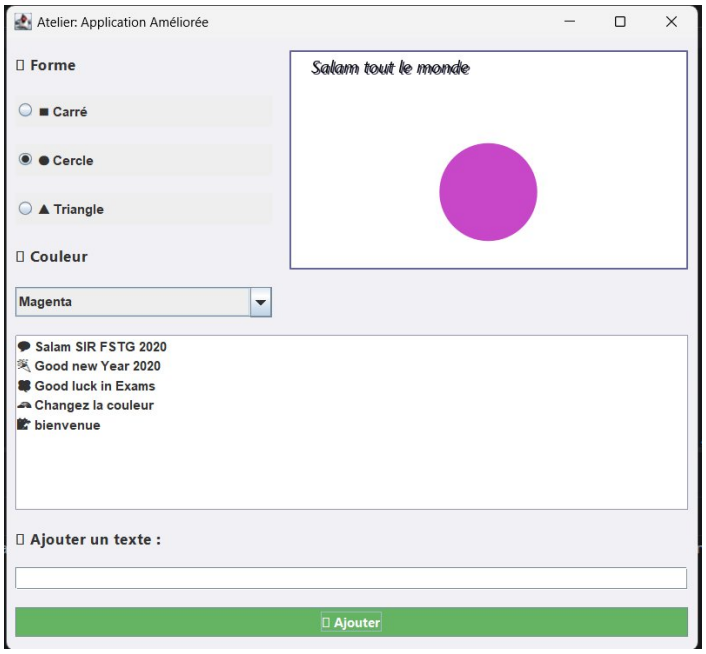

Figure 25 – Cercle Magenta, message italic, liste avec émojis, bouton stylisé

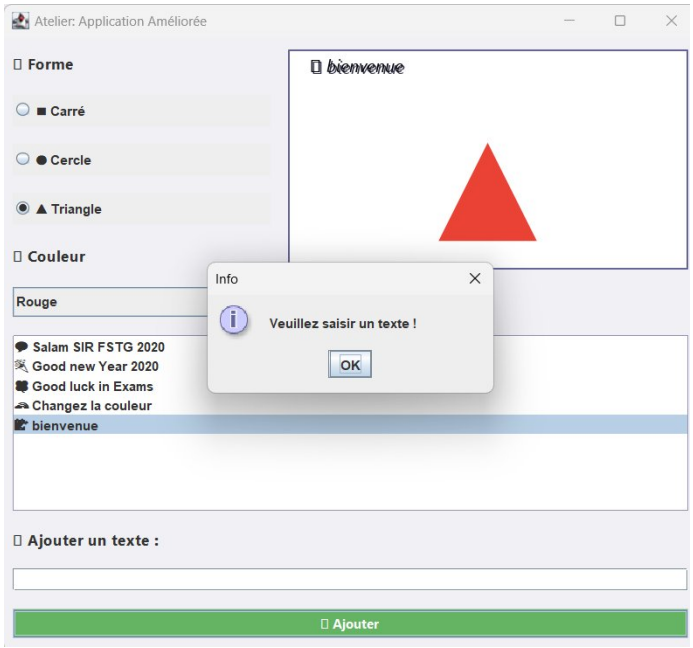

Figure 26 – Triangle rouge + dialog JOptionPane « Veuillez saisir un texte ! »

7. Conclusion

Ce laboratoire a permis d'acquérir une maîtrise progressive du développement d'interfaces graphiques en Java Swing, en partant d'une version de base jusqu'à une application enrichie et esthétique. Les cinq questions ont couvert :

- La **conception de fenêtres JFrame** avec GridBagLayout
- L'utilisation des **composants Swing** : JRadioButton, JComboBox, JList, JTextField, JButton
- La **programmation événementielle** avec ActionListener et ListSelectionListener
- Le **dessin 2D personnalisé** via paintComponent et Graphics/Graphics2D
- La **refactorisation lambda** pour un code concis et moderne (Java 8+)
- Les **améliorations visuelles** : antialiasing, couleurs RGB, tooltips, formes arrondies, polices

personnalisées et dialogs de validation

*Rapport réalisé par **Zakia Azizi** – Faculté des Sciences et Techniques, Université Cadi Ayyad, Marrakech – Module IHM, Lab4*